

Exam-Like Question

Abstract Data Types: *Hash Maps*

De Olla (25 points)

Let's say that money doesn't matter any more and I finally get to live the dream and open a vinyl café full of plants that serves only **café de olla** and some other Mexican treats. As part of my customer order system, I would like to keep an ordered queue of pending customer orders (strings representing the customers' names) together with the number of coffee cups that they ordered.

My system will be pretty flexible, so I need to be able to:

- Insert orders at the end of the queue,
- Cancel any order,
- Reposition any order immediately after another order, and
- Query a customer's order's cup count.

We'll implement all of this functionality in an augmented data structure called **CafeDeOllaQueue**, which supports the following interface:

Operation	Explanation	Target Runtime
cdoq = CafeDeOllaQueue()	Creates an empty order queue.	$\theta(1)$
len(cdoq)	Returns the number of pending orders.	$\theta(1)$
	Where name is the name of the customer and cups is the number of cups that customer ordered.	
cdoq.add(name, cups)	– If name does not exist in the order queue, insert a new order at the back of the queue, storing both the customer's name and the number of cups. – If name already exists, simply replace its stored cup amount with cups cups, without changing its position.	$\theta(1)$
cdoq.get_cups(name)	Returns the cup amount stored for this customer's name . Raise an Exception if the name is not in the queue.	$\theta(1)$
cdoq.prioritise(name, prev_name)	Moves the name 's order so it appears immediately after the (existing) order with the name prev_name . Both customers must already be in the queue, otherwise raise an Exception .	$\theta(1)$
cdoq.cancel(name)	Cancel the order associated with name . Raise Exception if this order doesn't exist.	$\theta(1)$
cdoq.serve(first=1)	Returns a Python list containing the first first orders in the order queue. If first $\geq$ current length, return a list with	$\theta(\text{first})$

Operation	Explanation	Target Runtime
	all the orders that are in the queue (both the names and the cup amounts). <code>serve</code> should also remove these orders from <code>cdoq</code> .	

To be clear:

- Aside from `serve`, all `CafeDeOllaQueue` operations must run in $\theta(1)$ amortised average time. This means that a sequence of `n` operations should complete in $\theta(n)$ average time.
- The `serve(first=1)` operation should run in $\theta(\text{first})$ time.

For example, you should expect the following interaction (the angle-bracket comments show the queue's front-to-back contents *after* each operation):

```
cdoq = CafeDeOllaQueue()           # < >
cdoq.add("Gloria", 1)               # <(Gloria, 1)>
cdoq.add("Maria", 3)               # <(Gloria, 1), (Maria, 3)>
cdoq.add("Vincy", 2)               # <(Gloria, 1), (Maria, 3), (Vincy, 2)>

print(cdoq.get_cups("Maria"))      # 3

cdoq.add("Gloria", 2)               # <(Gloria, 2), (Maria, 3), (Vincy, 2)>

print(cdoq.get_cups("Benny"))      # raises Exception

cdoq.prioritise("Maria", "Vincy")  # <(Gloria, 2), (Vincy, 2), (Maria, 3)>

served = cdoq.serve(2)             # <(Maria, 3)>
print(served)                      # [("Gloria", 2), ("Vincy", 2)]
```

Note: You can use any suitable data structures from the course, such as `ArrayQueue`, `ArrayStack`, `DoublyLinkedList`, `HashTableMap`, etc., to achieve the required functionality.